ESP32 Digital Twin Platform

Architecture Design — Version 5.0
Cupola Native Overlays, Closed-Loop Material Dispensing, Split ON/OFF Actuator Links

This version builds on v4 (Mosquitto local broker, unchanged here) and adds three new capabilities requested after the live demo: (1) Cupola hotspots showing the live value directly as a text overlay in the 360 scene, matching the existing label style already used for icons; (2) an automated, closed-loop material dispensing feature using a load cell + servo motor per material, driven by a target-weight input on the dashboard; and (3) actuator icons split into two separate hotspots — one for ON, one for OFF — each pointing to its own dedicated URL.

Field	Value
Document	ESP32 Digital Twin Platform — Architecture v5
Supersedes	ESP32_Digital_Twin_Architecture_v4 (Mosquitto local broker)
Broker	Mosquitto — self-hosted, local network (unchanged from v4)
New in this version	Cupola inline value overlays, load-cell+servo material dispensing, split ON/OFF actuator URLs
Data policy	Live-only, in-memory, no history/DB (unchanged)

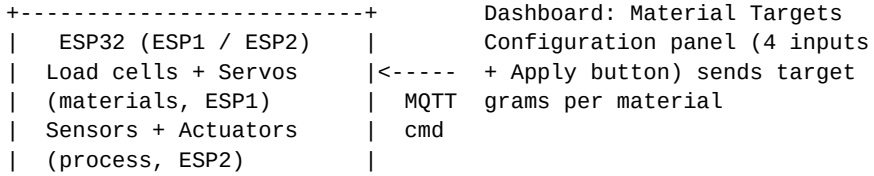
1. What Changed From v4

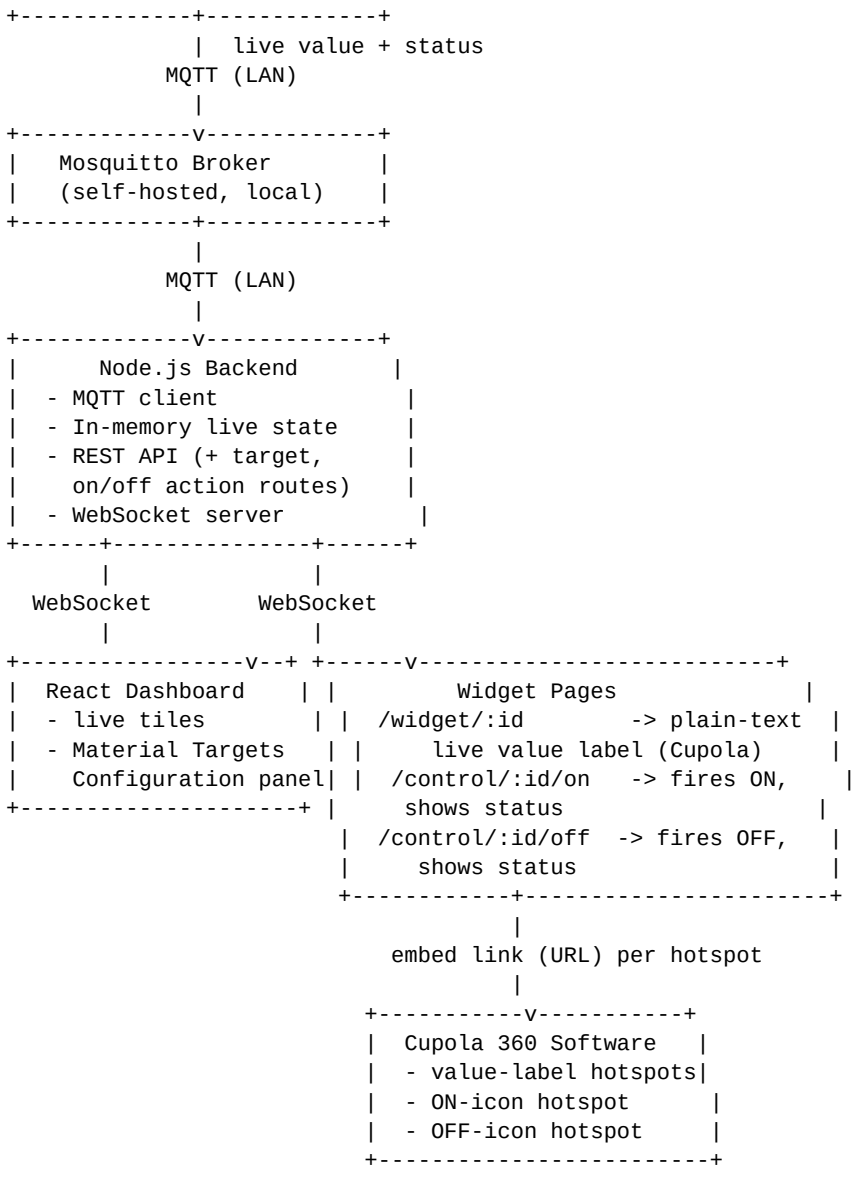
The broker, backend framework, and no-history data policy are unchanged from v4. The changes in this version are additive, driven directly by the live demo screenshots: Cupola hotspots now render the live value as a plain text label (not a styled card), a new dispensing control loop lets a target weight be entered once and reached automatically, and each actuator now has two independent hotspot links instead of one clickable widget.

v4 (old)	v5 (this document)
Widget page rendered as a styled card/tile inside Cupola's widget	Widget page renders as a plain text label (e.g. "iron_ore: 52 gm") matching C
Materials only ever displayed the live load-cell reading	Materials can now be given a target weight; ESP32 dispenses automatically v
Single control widget page per actuator, with an ON/OFF button inside it	Two separate hotspot URLs per actuator — one fires ON, the other fires OFF
No dashboard input for target values	New "Material Targets Configuration" panel on the dashboard, one input per m

2. High-Level Architecture (Updated)

The overall shape from v4 is unchanged — ESP32 to Mosquitto to Node.js backend to Dashboard / Widget pages to Cupola. Two things are added to the data flow: a target-driven command path for dispensing, and a pair of trigger-on-load pages for actuator ON/OFF.




live value + status
MQTT (LAN)
Mosquitto Broker
(self-hosted, local)
MQTT (LAN)
Node.js Backend
- MQTT client
- In-memory live state
- REST API (+ target, on/off action routes)
- WebSocket server
WebSocket
WebSocket
React Dashboard
- live tiles
- Material Targets
Configuration panel
Widget Pages
/widget/:id -> plain-text
live value label (Cupola)
/control/:id/on -> fires ON,
shows status
/control/:id/off -> fires OFF,
shows status
embed link (URL) per hotspot
Cupola 360 Software
- value-label hotspots
- ON-icon hotspot
- OFF-icon hotspot

NEW 3. Feature A — Cupola-Native Inline Value Overlay

In the current demo (the reference screenshot with "iron_ore: 0 gm", "limestone: 0 gm", "clay: 0 gm", "sand: 0 gm" floating next to their icons), Cupola is already showing values as plain floating text labels directly in the 360 scene — not as a bordered card or dashboard-style tile. The widget page behind each hotspot must be re-designed to match that exact visual language.

Design change

- No card, no border, no background box — transparent HTML page with a single line of text.
- Text format stays exactly "item_name: value unit" (e.g. "iron_ore: 52 gm"), same as the tile label already used elsewhere in the twin, so it visually blends with Cupola's own icon labels.
- Still a live page (WebSocket-driven) — the text updates in place the instant a new MQTT value arrives; no page reload needed.
- The same /widget/:id endpoint is reused for every material and sensor — only its rendering is stripped down to plain text output, no separate page per item type.

> No backend or MQTT changes are needed for this feature — it is purely a widget-frontend rendering change (HTML/CSS simplification of widget.html), reusing the exact same WebSocket subscription and REST snapshot call already defined in v3/v4.

NEW 4. Feature B — Automated Material Dispensing (Load Cell + Servo, Closed-Loop)

Each material feed (limestone, clay, iron_ore, sand) gets its own load cell (already reporting live weight, as in v3/v4) and its own servo motor that physically doses material. A target weight, entered once on the dashboard, is sent to the ESP32, which then runs a closed control loop locally: rotate the servo to dispense, keep reading the load cell, and stop — returning the servo to its home/back position — the instant the live weight reaches the target. This matches the target-input panel and the before/after weight screenshots from the demo.

4.1 Control Loop

Step	What Happens	Where
1	User enters a target (grams) for one or more materials and clicks "Apply Material Targets"	Dashboard
2	Dashboard sends one request with all target values to the backend	Dashboard -> Backend (REST)
3	Backend publishes a target command per material to its MQTT command topic	Backend -> Mosquitto -> ESP1
4	ESP1 rotates that material's servo to start dispensing	ESP1 firmware
5	ESP1 continuously reads the load cell and keeps publishing the live weight, exactly as it already	ESP1 -> Mosquitto -> Backend -> Dashboard/Widg
6	ESP1 compares the live reading to the target on every loop iteration	ESP1 firmware (local, no round-trip to backend ne
7	Once live weight >= target, ESP1 stops the servo and returns it to the home/back position	ESP1 firmware
8	ESP1 publishes a final "done" status for that material	ESP1 -> Mosquitto -> Backend
9	Backend marks the item as settled; dashboard tile and Cupola widget both show the new value (e.g. iron_ore: 52 gm)	Backend -> Dashboard/Widget

> The compare-and-stop decision (steps 6-7) runs on the ESP32 itself, not on the backend. This keeps the loop fast and reliable even if Wi-Fi briefly drops, and matches the live-only / no-history design philosophy — the backend never needs to store or drive the control logic, it only relays the target once and observes the resulting live values.

4.2 New MQTT Topics

Topic (example)	Direction	Payload	Purpose
plant/esp1/<material>	ESP1 -> Backend	{ "value": 52, "unit": "g" }	Existing live weight stream (unchanged)
plant/esp1/<material>/target/cmd	Backend -> ESP1	{ "target": 50 }	New: sets the target and starts dispensing
plant/esp1/<material>/target/status	ESP1 -> Backend	{ "status": "dispensing" | "done" }	New: lets backend/dashboard show progress and fi

4.3 New REST Endpoint

```
POST /api/materials/targets
{ "limestone": 50, "clay": 50, "iron_ore": 50, "sand": 50 }

-> Backend publishes one target/cmd message per material listed
-> Response: { "accepted": ["limestone","clay","iron_ore","sand"] }

// Optional single-item variant, if a hotspot or other UI needs to set just one target:
POST /api/item/:id/target
{ "target": 50 }
```

4.4 Dashboard: Material Targets Configuration Panel

A new panel on the dashboard, below the live sensor tiles, exactly as in the reference screenshot: one numeric input per material (Limestone Target (g), Clay Target (g), Iron Ore Target (g), Sand Target (g)) and a single "Apply Material Targets" button that sends all four values in one call to POST /api/materials/targets. This is a dashboard-level control — it is not exposed as an individual Cupola hotspot, since it needs all four fields visible together.

4.5 ESP32 Firmware Additions

- Servo motor driver per material feed on ESP1 (4 servos total: limestone, clay, iron_ore, sand)
- On receiving a target/cmd message: start rotating the corresponding servo to dispense
- Continue reading the load cell in the existing sensor loop and keep publishing live weight as before
- Local comparison: current load-cell reading vs. the received target, checked on every loop cycle
- Once reached: stop the servo, return it to the home/back position, publish a target/status = "done" message

NEW 5. Feature C — Split ON / OFF Actuator Links

In v3/v4, an actuator (e.g. motor_feed) had one control widget page with an ON/OFF button that the user tapped inside the embedded page. The reference screenshot instead shows two distinct icons per actuator directly in the Cupola scene — a green ON icon and a red OFF icon — each acting as its own hotspot. Since a Cupola hotspot simply opens a URL when clicked (there is no follow-up tap inside most embed types), each icon now needs its own dedicated URL that fires the action immediately when the page loads.

Design change

- Every actuator (motor_feed, and any other actuator such as the pulley/heater icons seen in the demo) gets two widget URLs instead of one: /control/:id/on and /control/:id/off.
- Opening /control/:id/on immediately sends the ON command to the backend (no button tap required inside the page) and then shows a short status line, e.g. "MOTOR_FEED: ON".
- Opening /control/:id/off works the same way for OFF.
- In Cupola, the green power icon's hotspot is configured with the .../on URL, and the red power icon's hotspot is configured with the .../off URL — exactly like configuring two separate YouTube/ThingsBoard embeds side by side today.

New REST Endpoints

```
GET /api/item/:id/action/on
-> Backend publishes { "command": "on" } to the actuator's command topic
-> Page shows: "<ITEM_NAME>: ON"

GET /api/item/:id/action/off
-> Backend publishes { "command": "off" } to the actuator's command topic
-> Page shows: "<ITEM_NAME>: OFF"

// The existing generic endpoint from v3/v4 remains available for the dashboard,
// where a real button + click handler is still the natural interaction:
POST /api/item/:id/action
{ "command": "on" | "off" }
```

> GET is used for the two new Cupola-facing endpoints because the action needs to fire purely from the hotspot opening a URL, with no button click available inside the page. This is convenient for a demo on a private LAN; because a GET request that changes state can be triggered by anything that loads the URL (including being pre-fetched or shared by accident), Section 9 (Future Scope) flags this for a confirmation step or authentication before this goes into a production/public setting.

6. Updated Cupola Hotspot -> URL Mapping

This table is what actually gets pasted into each Cupola hotspot's embed field. Materials and sensors use the plain-text value widget; each actuator now needs two separate hotspots.

Cupola Icon	Item	Type	URL to paste into hotspot
Limestone icon	limestone	material (value label)	http://<host>/widget.html?id=limestone
Clay icon	clay	material (value label)	http://<host>/widget.html?id=clay
Iron ore icon	iron_ore	material (value label)	http://<host>/widget.html?id=iron_ore

Sand icon	sand	material (value label)	http://<host>/widget.html?id=sand
Raw material icon	raw_material	material (value label)	http://<host>/widget.html?id=raw_material
Kiln temp icon	kiln_temp	sensor (value label)	http://<host>/widget.html?id=kiln_temp
Kiln humidity icon	kiln_humidity	sensor (value label)	http://<host>/widget.html?id=kiln_humidity
Preheat temp icon	preheat_temp	sensor (value label)	http://<host>/widget.html?id=preheat_temp
Preheat humidity icon	preheat_humidity	sensor (value label)	http://<host>/widget.html?id=preheat_humidity
Pulley temp icon	pulley_temp	sensor (value label)	http://<host>/widget.html?id=pulley_temp
Vibration icon	vibration	sensor (value label)	http://<host>/widget.html?id=vibration
Green power icon	motor_feed (and other actuators)	actuator ON	http://<host>/control.html?id=motor_feed&action=on
Red power icon	motor_feed (and other actuators)	actuator OFF	http://<host>/control.html?id=motor_feed&action=off

7. Updated Item Registry (Data Model)

Materials gain a dispensing capability (servo + target); the on/off split is handled at the widget/route layer and does not require new registry fields beyond marking an item as an actuator, as before.

item_id	Type	Source	Unit	Dispensable (servo + target)
limestone	material	ESP1	g	Yes
clay	material	ESP1	g	Yes
iron_ore	material	ESP1	g	Yes
sand	material	ESP1	g	Yes
raw_material	material	ESP1	g	No (blended output, not individually dosed)
kiln_temp	sensor	ESP2	C	-
kiln_humidity	sensor	ESP2	%	-
preheat_temp	sensor	ESP2	C	-
preheat_humidity	sensor	ESP2	%	-
pulley_temp	sensor	ESP2	C	-
vibration	sensor	ESP2	/8	-
motor_feed	actuator	ESP2	on/off	- (has ON/OFF split URLs, see Section 5)

8. Technology Stack Additions

Concern	Choice
Servo control (ESP1)	ESP32Servo library (or equivalent PWM servo driver) — one channel per material feed
Load cell reading (ESP1, existing)	HX711 (or equivalent load-cell amplifier) — already in place for live weight; reused for the closed
Control loop	Simple threshold compare (current >= target) running in the ESP1 main loop; no PID needed unl
Widget page rendering (new)	Plain HTML + minimal inline CSS, transparent background, single text node updated over WebS
Actuator trigger pages (new)	Plain HTML page that fires a fetch() to the REST action endpoint on window load, then displays t

9. Running It End-to-End — What's Different From v4

Steps 1-5 from v4 (Mosquitto, ESP32 publishing, backend, dashboard, widget server) are unchanged. Add the following checks before configuring Cupola:

- Test the dispensing loop standalone first: call POST /api/materials/targets with a small target (e.g. 10g) for one material, and confirm the servo runs, the live weight tile climbs, and the servo returns home once the target is hit — do this before wiring it into the dashboard button.

- Test each new ON/OFF URL directly in a browser tab (e.g. http:///control.html?id=motor_feed&action;=on) and confirm the actuator responds immediately on page load, before pasting the URL into Cupola.
- Open /widget.html?id=iron_ore on its own and confirm it now renders as a plain text label (no card/border), matching the look of Cupola's own icon labels, before embedding it as a hotspot.
- In Cupola, configure the green/red power icons as two separate hotspots per actuator, each with its own URL from the table in Section 6.

10. Future Scope (Updated)

- Authentication for dashboard and widget endpoints (carried over from v4)
- Add a confirmation step or short-lived token to the GET-triggered ON/OFF URLs before any public/production deployment, since a plain GET link can be triggered unintentionally
- Overshoot/undershoot tuning for the dispensing loop (e.g. a small PID or slow-down-near-target ramp) if the simple threshold stop proves imprecise in testing
- PLC integration (Modbus / OPC UA) alongside ESP32
- Support for multiple ESP32 devices / multiple plant lines
- Role-based access control for actuator commands and target changes
- Optional historical logging layer, kept separate from the live path
- Optional migration path back to a managed broker (HiveMQ, EMQX Cloud, AWS IoT Core) if remote, multi-site access becomes a requirement later

---

End of document — ESP32 Digital Twin Platform, Architecture v5.0 (Cupola inline overlays, closed-loop dispensing, split ON/OFF actuator links).